

```
=====
                        QUICKSORT ALGORITHM
                        in C Language
=====
```

```
PROGRAM DESCRIPTION:
```

QuickSort is a highly efficient divide-and-conquer sorting algorithm. It works by selecting a pivot element and partitioning the array into two subarrays: elements less than pivot and elements greater than pivot. The subarrays are recursively sorted.

```
=====
                        SOURCE CODE
=====
```

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);

    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int arr[100], n;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);

    printf("Sorted array:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
}
```

```

    }

    return 0;
}

```

PROGRAM EXPLANATION

1. SWAP FUNCTION:


```
void swap(int *a, int *b)
```

 - Exchanges values of two integers using pointers
 - Uses temporary variable to avoid data loss
 - Called by partition function to reorder elements
2. PARTITION FUNCTION:


```
int partition(int arr[], int low, int high)
```

 - Selects last element as pivot (arr[high])
 - Uses two pointers approach:
 - * i: Pointer to smaller elements
 - * j: Pointer for comparison
 - Moves smaller elements to the left
 - Places pivot in its correct position
 - Returns pivot's final position
3. QUICKSORT FUNCTION:


```
void quickSort(int arr[], int low, int high)
```

 - Recursive function for sorting
 - Base case: if low >= high, array is sorted
 - Recursive case:
 - * Partitions array
 - * Recursively sorts left subarray (low to pi-1)
 - * Recursively sorts right subarray (pi+1 to high)
4. MAIN FUNCTION:
 - Takes input array from user
 - Calls quickSort with range 0 to n-1
 - Prints sorted array

ALGORITHM STEPS

QuickSort Process:

- Step 1: Select pivot element (usually last element)
- Step 2: Partition array around pivot
 - Elements < pivot go to left
 - Elements > pivot go to right
- Step 3: Recursively sort left partition
- Step 4: Recursively sort right partition
- Step 5: Combine (array is already sorted in place)

Partition Steps:

1. Set pivot = arr[high]
2. Initialize i = low - 1
3. For each element j from low to high-1:
 - If arr[j] < pivot:
 - * Increment i
 - * Swap arr[i] and arr[j]
4. Swap arr[i+1] with pivot arr[high]
5. Return i+1 (pivot position)

EXAMPLE WALKTHROUGH

Array: [64, 34, 25, 12, 22, 11, 90]

Goal: Sort in ascending order

Initial Call: quickSort(arr, 0, 6)

Iteration 1 - Partition (low=0, high=6):

Pivot = 90

Array after partition: [64, 34, 25, 12, 22, 11, 90]

Pivot index (pi) = 6

Iteration 2 - Recursively sort left (0 to 5):

Partition (low=0, high=5):

Pivot = 11

Array: [11, 34, 25, 12, 22, 64, 90]

Pivot index = 0

Continue recursively...

Left: [] (empty)

Right: [34, 25, 12, 22, 64] (partition and sort)

Continuing recursively until all subarrays are single elements or empty.

Final Sorted Array: [11, 12, 22, 25, 34, 64, 90]

COMPLEXITY ANALYSIS

Time Complexity:

Best Case:	$O(n \log n)$	- Pivot divides array evenly
Average Case:	$O(n \log n)$	- Typical random input
Worst Case:	$O(n^2)$	- Pivot always smallest or largest (can be avoided with random pivot)

Space Complexity:

$O(\log n)$ - Recursion call stack depth

Comparison with Other Sorting Algorithms:

Algorithm	Best Case	Average Case	Worst Case	Space
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
HeapSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
BubbleSort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
InsertionSort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$

ADVANTAGES & DISADVANTAGES

Advantages:

- Very efficient for large datasets ($O(n \log n)$ average)
- In-place sorting (doesn't need extra array)
- Cache-friendly (good locality of reference)
- Widely used in standard libraries
- Can be optimized with different pivot strategies

Disadvantages:

- Worst-case $O(n^2)$ time complexity

- â€ Not stable (relative order of equal elements may change)
- â€ Recursive (requires stack space)
- â€ Performance depends on pivot selection
- â€ Quicksort of already sorted data is $O(n^2)$

PIVOT SELECTION STRATEGIES

1. Last Element (Current Implementation):
Simple but can lead to $O(n^2)$ on sorted arrays
2. First Element:
Similar issues to last element
3. Middle Element:
Better than first/last but still not optimal
4. Random Element:
Reduces probability of worst case
Better for nearly sorted arrays
5. Median-of-Three:
Choose median of first, middle, last
Provides good average performance

OPTIMIZATION TECHNIQUES

1. Random Pivot Selection:
 - Select random element as pivot
 - Reduces worst-case probability
2. Three-Way Partitioning:
 - Handle duplicate elements efficiently
 - Creates three partitions: less, equal, greater
3. Hybrid Approach:
 - Use QuickSort for larger subarrays
 - Switch to InsertionSort for small subarrays
4. Tail Call Optimization:
 - Sort smaller partition first
 - Use iteration for larger partition

PRACTICAL APPLICATIONS

1. Database Indexing
2. Search Engine Ranking
3. Operating System Process Scheduling
4. Graphics Applications (pixel sorting)
5. Network Packet Sorting
6. File System Operations
7. Data Analysis and Statistics